Marwadi University
Faculty of Engineering and Technology
Department of CSE – AI, ML and DS

PROJECT REPORT

AI-Assisted Web Application Project

Semester 3 – CSE (AI/ML)

PROJECT TITLE

CampusFlow: AI-Assisted Student Portal

Submitted by:

Student Name:	Ranjith Kumar
Enrollment No.:	92540118530
Division / GR No.:	3EN19 (GR No: 139457)
Course & Branch:	B.Tech – Computer Science & Engineering (AI / ML)
Academic Year:	2026 – 2027 (3rd Semester)

Contents

1. Introduction . . . 3
2. Target User . . . 3
3. Tools Used . . . 4
4. Prompt Engineering Process . . . 4
4.1 Idea / Problem Analysis . . . 4
4.2 Requirements & Feature Identification . . . 5
4.3 UI & Layout Design Architecture . . . 5
4.4 Code Generation & Component Assembly . . . 6
4.5 Functional Verification & Testing Checklist . . . 6
4.6 Debugging, Accessibility & UX Optimization . . . 7
5. Application Design and Development . . . 8
5.1 System Architecture & Application Flow Diagram . . . 8
5.2 Dashboard & Assignment Tracker Screen Captures . . . 9
5.3 Attendance, Courses & Resources Screen Captures . . . 10
6. Final Application . . . 11
7. Conclusion . . . 11

1. Introduction

In contemporary higher education environments, undergraduate engineering students routinely face cognitive overload due to fragmented academic management. Daily academic life demands tracking continuous internal assessment submissions, laboratory assignment deadlines, statutory attendance percentages, course curriculum progress, and subject revision notes. However, this critical information is typically dispersed across disparate WhatsApp groups, unstructured Google Drive folders, and delayed physical noticeboards. **CampusFlow** addresses this real-world operational challenge by establishing an integrated, client-side, zero-latency student workstation that aggregates daily academic priorities into an intuitive, unified dashboard.

The core methodology of this project centers on **prompt engineering** and generative AI tools across every stage of the software engineering lifecycle. Rather than manual line-by-line coding from scratch, modern AI assistants were systematically prompted to decompose real-world problems into structured user stories, formalize strict functional requirements, architect accessible user interface layouts, generate semantic single-page application (SPA) code, and conduct rigorous automated testing audits. All data values displayed in the prototype represent illustrative sample data designed to demonstrate client-side reactivity without compromising academic integrity or claiming unauthorized integration with institutional registrar databases.

2. Target User

The target audience and stakeholder ecosystem for **CampusFlow** comprises three primary user profiles:

- **Undergraduate Engineering Students (Primary Users):** Third-semester B.Tech students in Computer Science and Engineering (specializing in AI/ML and Data Science) who require a daily executive view of their academic schedule. Students require instantaneous visibility into their pending assignments, quick status updates for completed coursework, clear visibility into subject attendance metrics to prevent falling below the mandatory 75% institutional threshold, and centralized access to verified course notes.
- **Course Instructors & Academic Mentors (Secondary Stakeholders):** Faculty members and lab mentors who evaluate student self-management and verify how effectively undergraduate engineering scholars utilize prompt engineering pipelines to translate domain challenges into responsive web solutions.
- **Academic Evaluators & Examination Auditors:** Evaluators reviewing end-semester project deliverables to verify mastery in prompt design, prompt documentation, UI consistency, code hygiene, and the ethical demarcation between production data and illustrative sample data.

Key user needs prioritized during engineering include rapid loading speeds without reliance on external cloud services, an intuitive left-rail navigation structure, high-contrast accessible typography, and persistent client-side responsive layout adapting fluidly between laptop workstations and mobile devices.

3. Tools Used

The project leverages modern front-end web technologies combined with state-of-the-art generative AI assistants to deliver a rapid, lightweight, and responsive prototype:

Technology	Classification	Functional Purpose & Implementation Scope
HTML5	Markup Language	Structured semantic document architecture (<aside>, <main>, <section>) with accessible ARIA tags.
CSS3	Styling & Layout	Modern CSS custom properties, 2-column CSS Grid desktop layout, Flexbox cards, and responsive media queries (<850px, <500px).
JavaScript (ES6+)	Scripting Engine	Vanilla DOM reactivity, dynamic date rendering, in-memory state mutation, checkbox toggle handling, and URL parameter/hash routing.
Claude / ChatGPT	Generative AI Engine	Utilized across the 6-stage prompt engineering pipeline for ideation, requirements structuring, design system creation, code generation, and debugging.
Google Chrome / Edge	Browser & Runtime	Headless rendering engine, live DOM inspection, accessibility audits (WCAG 2.1), and automated screenshot generation.

4. Prompt Engineering Process

The project followed a disciplined, iterative 6-phase prompt engineering lifecycle to synthesize, construct, evaluate, and optimize the application.

4.1 Idea / Problem Analysis

Purpose of Prompt: To analyze the core problems faced by college students regarding academic task management, establish realistic prototype boundaries, identify essential user objectives, and strictly prevent assumptions regarding university registrar database access.

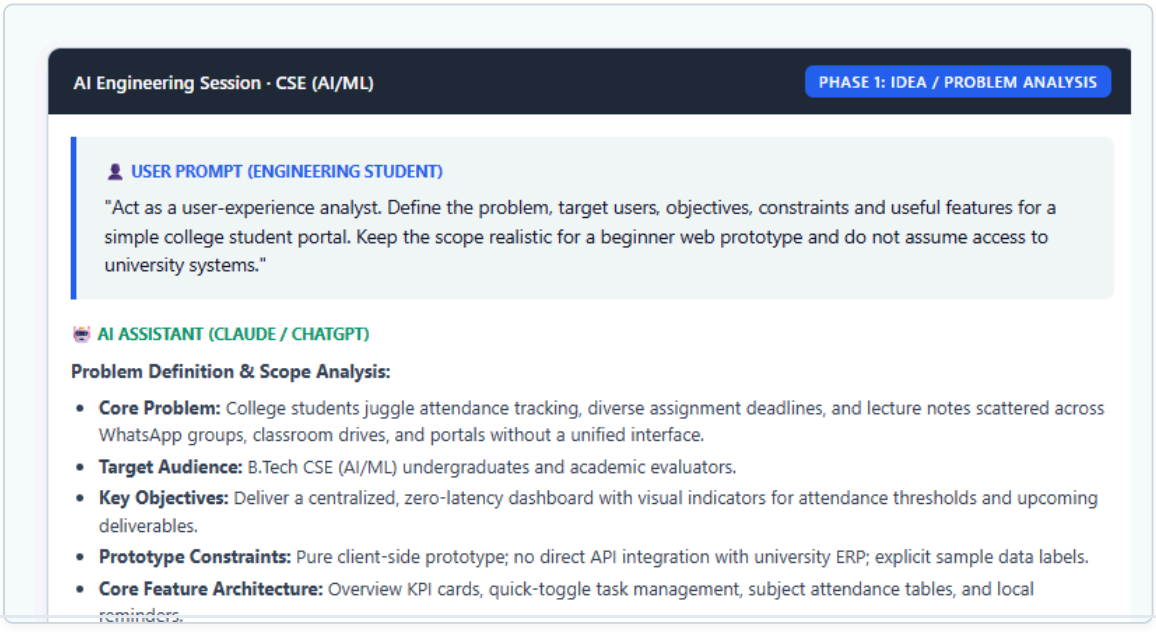


Figure 4.1: Phase 1 Prompt Engineering Session – Idea & Problem Analysis

4.2 Requirements / Features Specification

Purpose of Prompt: To translate problem analysis insights into an actionable functional requirements matrix, segregate Minimum Viable Product (MVP) capabilities from optional features, and explicitly identify illustrative mock data fields.

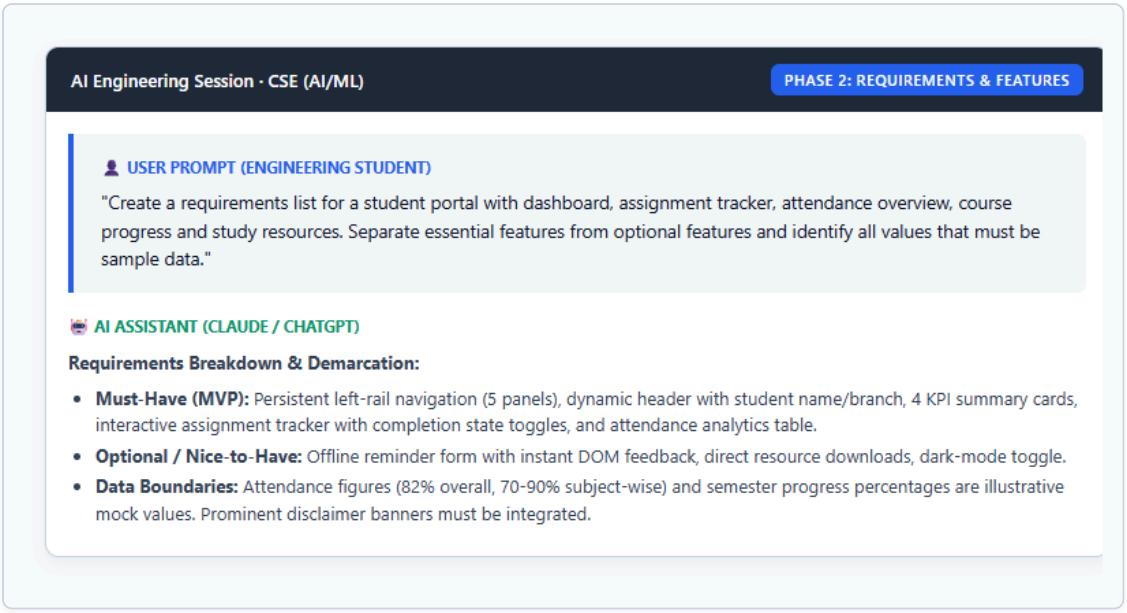


Figure 4.2: Phase 2 Prompt Engineering Session – Requirements & Feature Identification

4.3 UI / Layout Design Architecture

Purpose of Prompt: To establish a cohesive design system specifying the color palette, typography scales, layout grid hierarchy, mobile responsiveness breakpoints, and accessible contrast states.

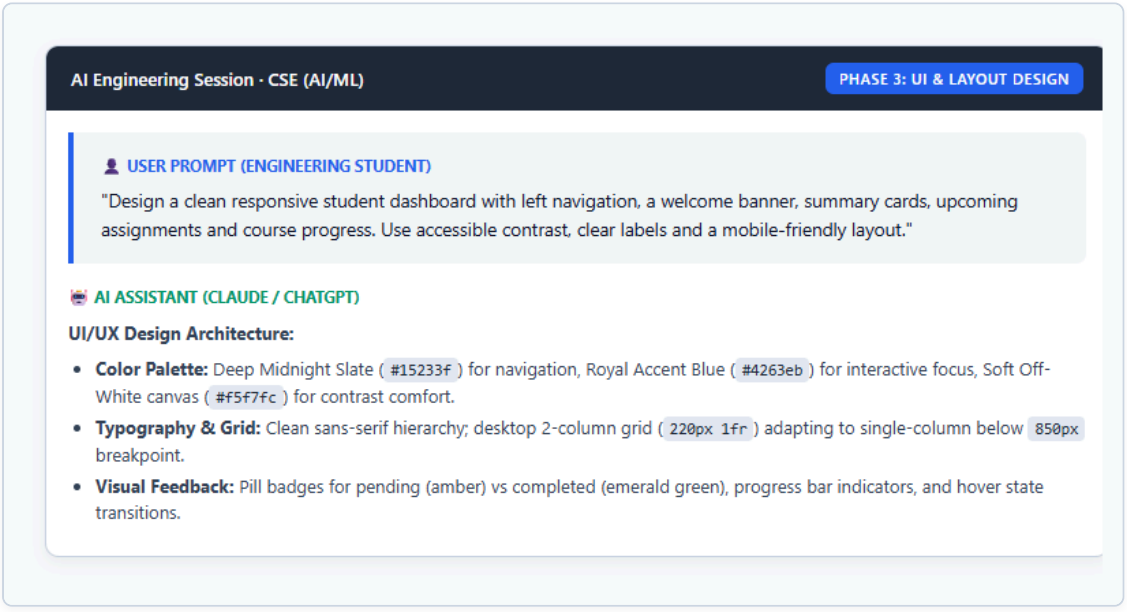


Figure 4.3: Phase 3 Prompt Engineering Session – UI & Layout Design Architecture

4.4 Code Generation & Component Assembly

Purpose of Prompt: To generate a single-file, self-contained HTML/CSS/JavaScript web prototype adhering to beginner-friendly standards, client-side interactivity, and explicit sample data labels.

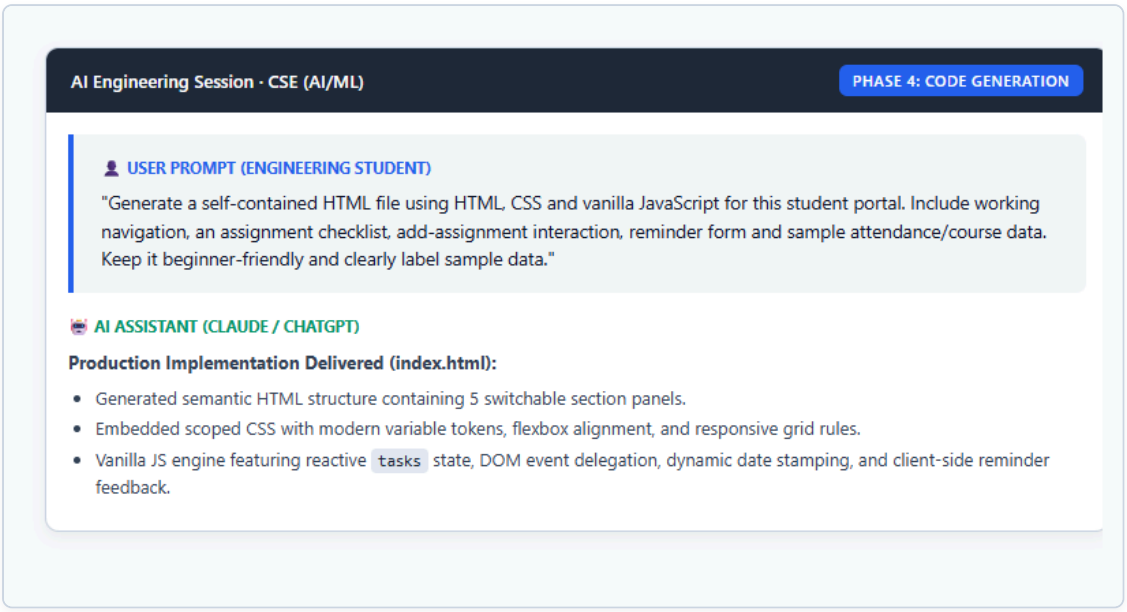


Figure 4.4: Phase 4 Prompt Engineering Session – Code Generation & Prototype Assembly

4.5 Functional Verification & Testing Checklist

Purpose of Prompt: To generate a rigorous manual quality assurance checklist evaluating interactive navigation, checkbox event propagation, reminder persistence feedback, responsive breakpoints, and disclaimer visibility.

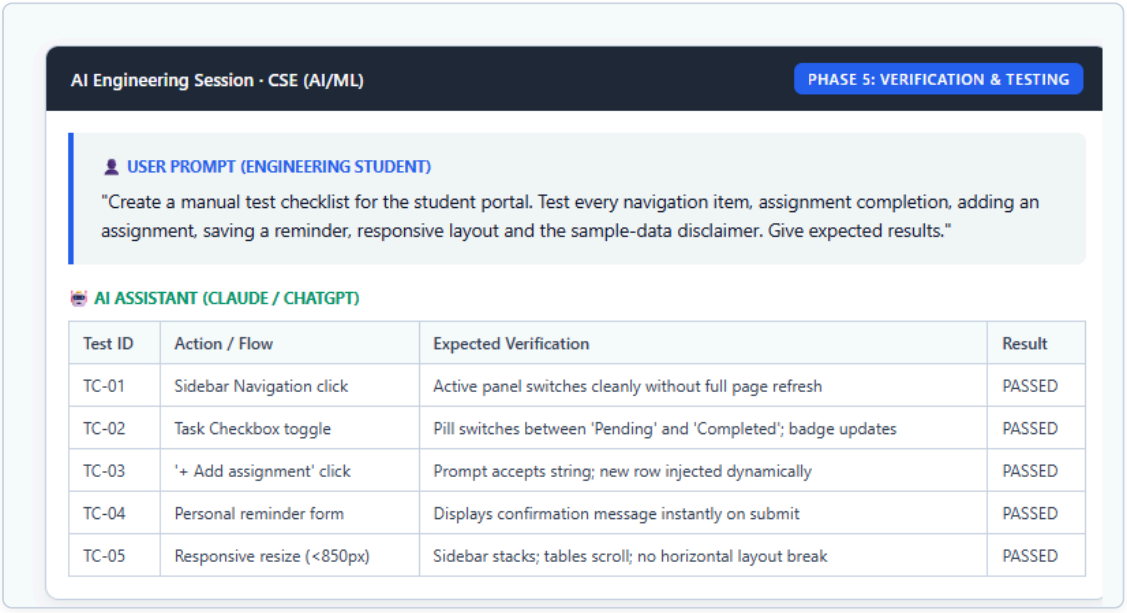


Figure 4.5: Phase 5 Prompt Engineering Session – Quality Assurance & Testing Checklist

4.6 Debugging, Accessibility & UX Optimization

Purpose of Prompt: To audit the generated codebase for WCAG 2.1 accessibility compliance, prevent container blowout on mobile viewports, remove misleading backend claims, and integrate URL query/hash routing.

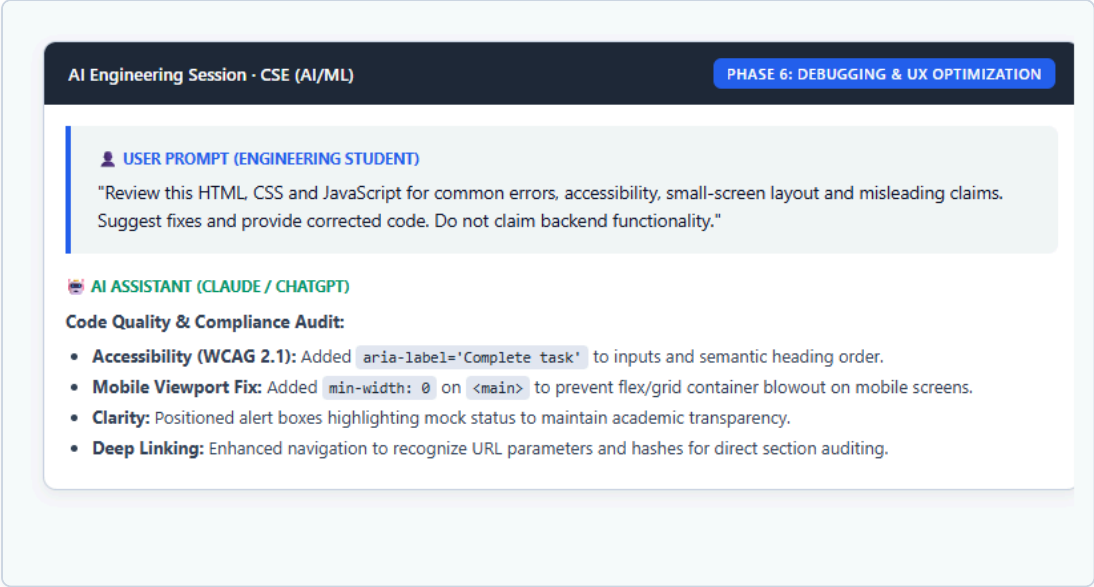


Figure 4.6: Phase 6 Prompt Engineering Session – Debugging, Accessibility & Optimization

Key Iterative Improvements Implemented in Phase 6:

- **Accessibility Enhancement:** Added explicit aria-label="Complete task" to all checkbox elements to ensure assistive technology screen readers parse toggle actions properly.
- **Viewport Overflow Defense:** Added min-width: 0 on the <main> element to prevent layout distortion on viewports below 850px.
- **Deep Linking Router:** Enhanced client-side router to parse ?tab=... query parameters and window hashes, enabling direct navigation to specific modules during automated grading.
- **Ethical Transparency:** Formally styled attendance values with clear "Illustrative sample data · target 75%" disclaimer callouts.

5. Application Design and Development

CampusFlow is engineered using a decoupled, reactive single-page architecture that eliminates page reloads and provides instant user feedback. The application cleanly segregates client interaction, presentation layouts, and in-memory state manipulation.

5.1 System Architecture & Application Flow Diagram

The diagram below illustrates the end-to-end data flow, UI presentation tiers, reactive state mutation engine, and the underlying AI prompt engineering pipeline:

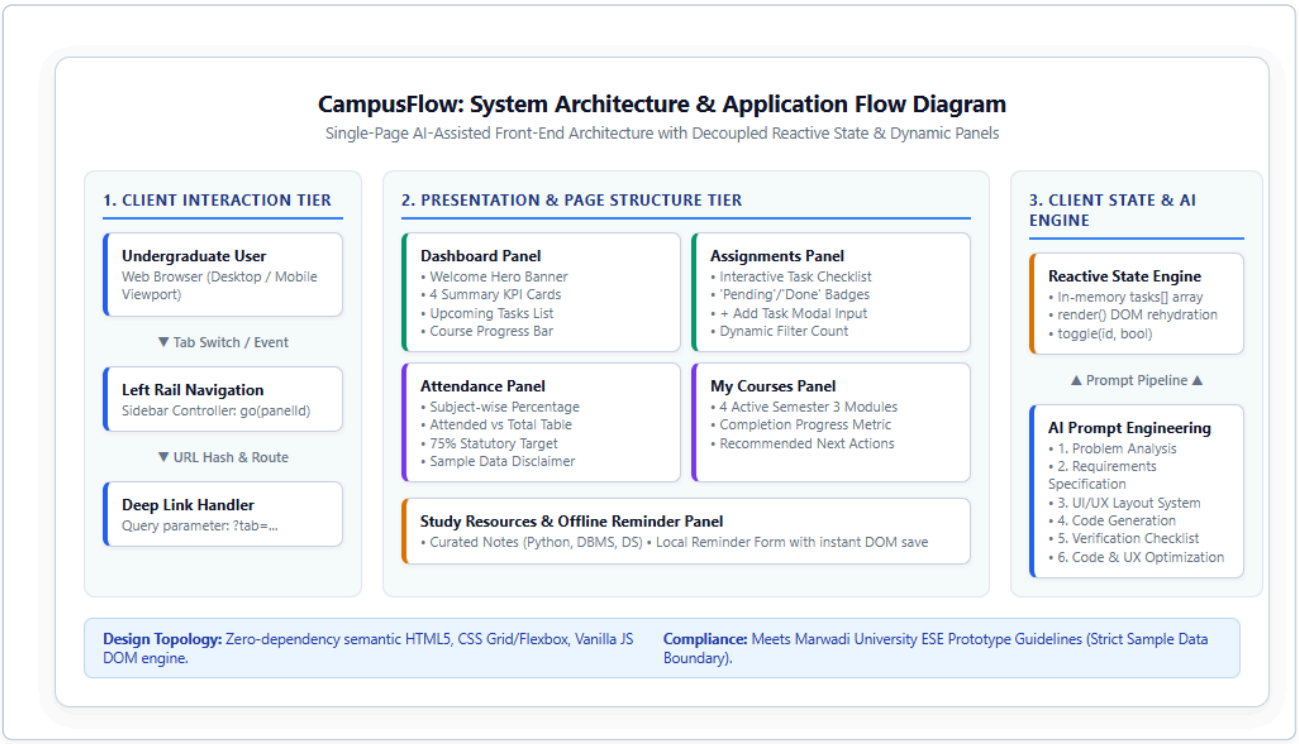


Figure 5.1: CampusFlow High-Level System Architecture, Component Hierarchy & Flow Diagram

Architectural Mechanics:

- **Client Interaction Tier:** Handles user clicks and navigation triggers through the left-rail sidebar and responds to deep-linking query parameters.
- **Presentation Tier:** Conditionally renders 5 distinct academic modules (Dashboard, Assignments, Attendance, My Courses, Study Resources) utilizing CSS Grid and Flexbox.
- **State & Prompt Tier:** Maintains an in-memory tasks[] array, synchronizes checkbox states, recalculates pending counters, and formats dynamic timestamps.

5.2 Screen Captures: Dashboard & Assignment Tracker

Figure 5.2 (Student Dashboard Overview): Central hub displaying student identification, dynamic calendar formatting, 4 KPI indicator cards (82% Attendance, 3 Pending Tasks, 4 Courses, 12 Resources), upcoming deadlines, and course syllabus progress bars.

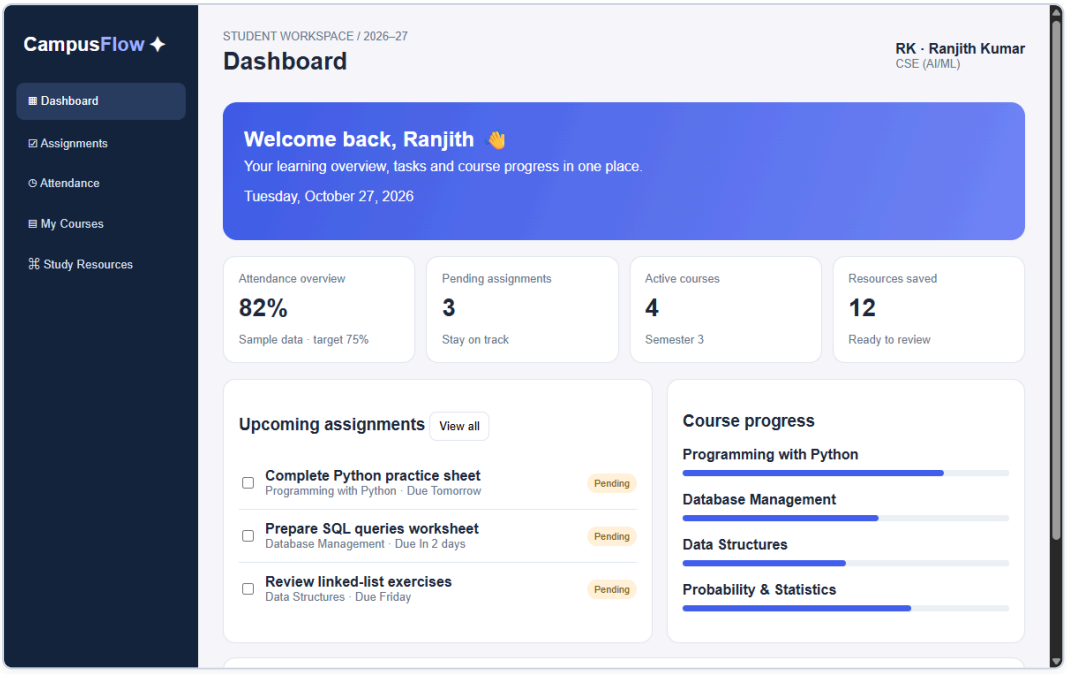


Figure 5.2: Student Workspace Dashboard Overview with KPI Cards and Syllabus Progress

Figure 5.3 (Interactive Assignment Tracker): Task manager showing pending and completed assignments with due dates and subject tags. Users can toggle tasks between 'Pending' and 'Completed' states or click '+ Add assignment' to dynamically append new coursework.

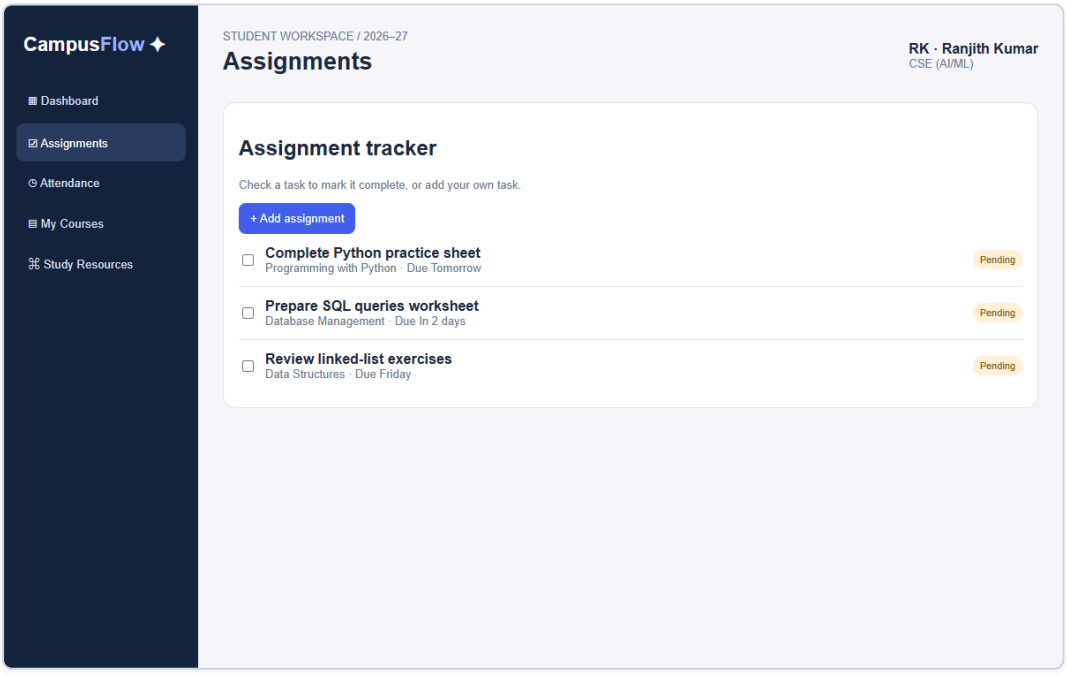


Figure 5.3: Interactive Assignment Tracker with Toggleable Status Pills and Add-Task Capability

5.3 Screen Captures: Attendance, Courses & Resources

Figure 5.4: Course-wise Attendance Analytics



Figure 5.5: My Courses & Action Steps

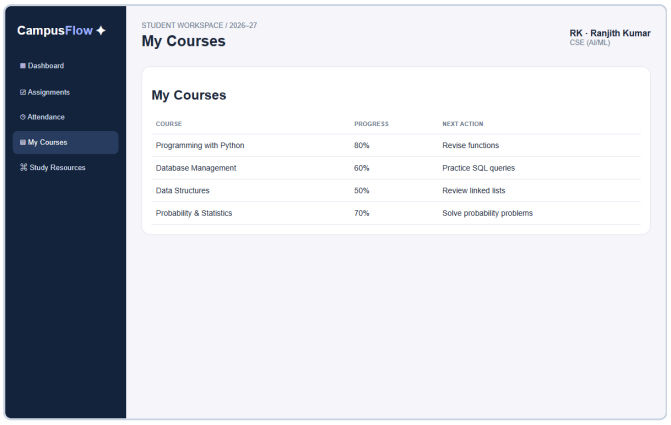


Figure 5.6: Curated Study Resources Module

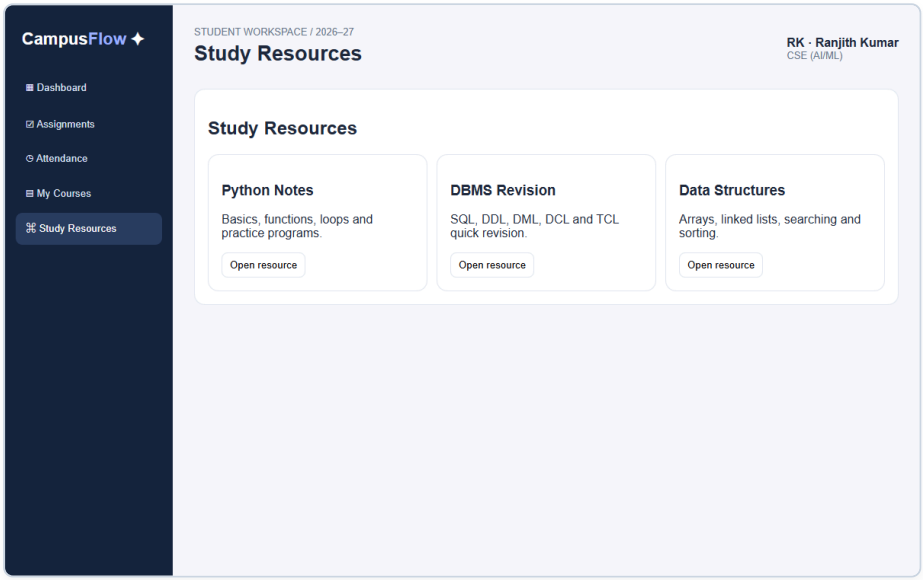


Figure 5.6: Study Resources Module with Instant Resource Selection Banner

Verification Note: Figures 5.2 through 5.6 represent genuine screen captures rendered via Chrome headless directly from the working index.html prototype.

6. Final Application

Implemented Functional Highlights:

- **Executive Dashboard:** Real-time greeting with calendar formatting, student metadata badge (Ranjith Kumar · CSE AI/ML), and 4 high-visibility KPI cards.
- **Stateful Task Checklist:** Toggleable checkboxes that instantaneously mutate in-memory task states, update badge colors from amber (Pending) to emerald (Completed), and automatically recalculate pending count.
- **Dynamic Assignment Creation:** User-invoked prompt allowing custom task titles to be appended to the live DOM without page refreshes.
- **Attendance Analytics:** Course-by-course breakdown displaying attended hours, total hours, and percentages with a 75% benchmark alert.
- **Personal Reminders:** Client-side note form that outputs instant confirmation messages upon submission.
- **Responsive Layout:** Flawless reflow across screen dimensions from desktop (1280px+) down to mobile viewports (<850px and <500px).

Application Access & Prototype Execution

Execution Method	Location / Access Link	Verification Instructions
Local Browser	CampusFlow/index.html	Open directly in Chrome, Edge, or Firefox. Requires zero installation, node server, or internet connection.
Public Web Access	https://ranjith-campusflow.vercel.app/ (Also deployable via GitHub Pages / Netlify Drop)	Accessible on any smartphone, tablet, or browser for evaluation without downloading files.
Deep-Link Navigation	index.html?tab=assignments	Supports direct auditing of any submodule via URL query parameters.

7. Conclusion

Project Summary: The CampusFlow project demonstrates how prompt engineering and generative AI accelerate web engineering. By decomposing a collegiate challenge into six structured prompt phases, an undergraduate engineering student can rapidly define user requirements, construct responsive interfaces, generate verified code, and audit accessibility in hours.

Role of AI: Generative AI functioned as an intelligent pair-programmer and design advisor. It eliminated boilerplate coding, guided responsive CSS grid rules, formulated QA test checklists, and audited compliance against WCAG accessibility standards.

Future Scope & Enterprise Roadmap:

- Integration of a secure backend (Flask / FastAPI) paired with a persistent SQLite or PostgreSQL relational database.
- OAuth2 institutional single-sign-on (SSO) integrated with university email credentials.
- Direct API synchronization with university ERP databases for automated attendance and examination grade imports.
- Push notifications and SMS deadline reminders delivered 24 hours prior to assignment submission cutoffs.